MAXX STEELE™

PROGRAMMER'S TECHNICAL REFERENCE GUIDE

RoboForce

Version 1.0 | May 24, 2024

Maxx Steele Technical Manual

Maxx-Steele-1984-Robot contributors

2026-06-25

Contents

1 Maxx Steele Technical Manual

This manual describes how to program the 1984 CBS Toys / Ideal **Maxx Steele** robot at the software level: bytecode motion programs, speech, music, cartridge images, and the 6502 internal ROM that interprets them.

It is modeled on the structure of the Commodore 64 Programmer's Reference Guide — numbered chapters, appendices, a quick-reference card, and a schematic index — adapted to Maxx Steele hardware.

PDF (latest): `Maxx-Steele-Technical-Manual.pdf` — front/rear covers plus chapters; rebuilt when `.md` or cover images (`cover-front.jpg`, `cover-rear.jpg`) change (`tools/build_technica` locally; GitHub Actions on push to `main`).

1.1 What this guide covers

Document	Audience	Focus
This guide	Programmers, reverse engineers	Bytecode language, memory map, I/O, internal ROM hooks
`Cartridge/PROGRAMMING.md`	Cartridge authors	Step-by-step cart layout, tools, demo walkthrough
`Chassis/Manual/`	Owners	Factory user / reference manuals (operation, not ROM layout)
`MechanicalManual/`	Repairers	Chassis disassembly, reassembly, teardown photos

Primary sources: R. Wind disassemblies (`maxx_internal_ROM.dsm`, `maxx_demo_ROM_532.dsm`), `tools/maxx_rom.py`.

1.2 How to use this reference guide

If you are new to Maxx programming, read in order:

1. Chapter 1 — Bytecode programming rules
2. Chapter 2 — Opcode vocabulary
3. Chapter 3 — Motion and display
4. Work through the demo program in `Cartridge/PROGRAMMING.md` §5

If you are writing cartridges, add:

5. Chapter 5 — Internal ROM operating system
6. Chapter 7 — UltraMaxx BASIC language
7. `Cartridge/PROGRAMMING.md` §9 (authoring workflow)

If you are reverse engineering hardware, use:

- Chapter 6 — Input/output guide
- Schematics
- Annotated `.dsm` listings under `Mainboard/Firmware/` and `Cartridge/Examples/`

Keep Quick reference and Appendices open while coding.

1.3 Applications guide

Application	Mode (`$0D`)	What you need
Remote control (immediate)	0	Remote keypad; opcodes executed live
Learn key sequences	1	Records into program RAM
Enter program steps	2	Keypad maps to opcodes via `$E6B5` table
Run stored program	3	Bytecode at `$0200`, terminator `FF FF`
Built-in games	4	`JMP ($0094)` → `$F8CE`; game 1/2 at `$F8E9` / `$FAA4`
Factory demo	3 + cart	`CBSDemo.532` bootstrap
Custom cartridge	3 + cart	4 KB EPROM image; see Ch 5–7 + cartridge manual
UltraMaxx BASIC authoring	3 + cart	Ch 7; `tools/maxx compile` → `.532`
Speech / music authoring	any	Ch 4; phrase tables in cart or RAM
Repair / RE	—	Ch 6, Schematics, `DataSheets/`

1.4 Firmware images in this archive

Image	Path
Internal 8 KB ROM	`Mainboard/Firmware/Binary/Maxxrom.64`
Demo cartridge (4 KB)	`Cartridge/Examples/CBSDemo/Firmware/Binary`
UltraMaxx cartridge (4 KB)	`Cartridge/Examples/UltraMaxx/Firmware/Bina`

Tools: `tools/maxx` — compile UltraMaxx BASIC, validate, list, upload; `tools/maxx_rom.py` — disassemble, template, opcode export.

2 Chapter 1 — Bytecode Programming Rules

This chapter describes the rules for writing Maxx Steele **programs**: how bytecode is stored, how operating modes affect entry, and practical techniques for building sequences.

The robot does not run arbitrary user 6502 code for motion demos. The internal ROM interprets a **bytecode program** in RAM. Cartridges supply tables and a short bootstrap stub; see Chapter 5.

2.1 Introduction

A Maxx program is a linear sequence of **(opcode, operand)** byte pairs. Each pair is one step. The executor reads from program RAM, displays the opcode name on the LED segment display, and performs the action.

Programs are edited through the remote keypad (Program mode), loaded from a cartridge bootstrap, or built offline and burned into EPROM.

2.2 Program storage

Property	Value
RAM region	`$0200–03FE`
Step size	2 bytes (opcode, operand)
Maximum steps	255 pairs (512 bytes); **FULL** displayed when `$037F` exceeded
Terminator	`FF FF` (required)

Example minimal program:

```
83 10    ; speak RAM phrase 0
0C 03    ; delay 3 seconds
01 10    ; drive forward
0B 00    ; home all joints
FF FF    ; end
```

2.3 Operating modes (`$0D`)

The mode byte in zero page selects how keypad input and the executor behave.

Value	Name	Behavior
`$00`	Immediate	Keys execute motion/speech instantly
`$01`	Learn	Records key sequences into program RAM
`$02`	Program	Enter bytecode steps via keypad
`$03`	Execute / Game	Run the stored program

Cartridge entry stubs typically initialize status bytes `$02` and `$03` before jumping to the internal main loop at `$E0B6`.

2.4 Status bytes

2.4.1 `$02` — Status A

Set by cartridge bootstrap (demo uses `LDA #$02` / `STA $02`). Bits include speech error (**SEr**), enable backoff (**Ebof**), power-down, drive, and speech flags. Keypad rows toggle subsets; see Appendix E.

2.4.2 `$03` — Status B

Demo entry uses `LDA #$82` / `STA $03` (**PDon**, **Edof**, **SPof**, **UCon**). Governs user control and execute gating.

2.5 Program pointers (zero page)

Addr	Role
`$0F` / `$10`	Current step in `$0200` table (program mode / execute)
`$11` / `$13`	Current opcode and operand during entry
`$24` / `$25`	Program byte pointer used by executor

2.6 Programming techniques

2.6.1 Delays

Opcode `$0C` operand is delay time in **seconds** (decimal). Chain delays between motion commands so mechanics can finish.

```
0C 02    ; wait 2 seconds
01 14    ; forward (operand scale empirical — see Ch 3)
0C 04    ; wait 4 seconds
```

2.6.2 Homing

Opcode `$0B` with operand **`$00` only** — returns joints to a known position before the next move.

2.6.3 Speech then motion

RAM phrases (`$83`) must be copied to `$0500` before execute (cartridge stub or prior setup). ROM phrases (`$82`) need no RAM table.

2.6.4 Program length

If program entry exceeds available RAM, the display shows **FULL** (internal ROM compares against `$037F`).

2.6.5 Marker opcode `$FE`

Display-only **beg** marker; does not affect execution flow in the demo cart.

2.7 Beginner checklist

1. Every program ends with `FF FF`.
2. Operands are one byte (`$00`–`FF`); meaning depends on opcode (Ch 2–4).
3. Test with `python3 tools/maxx_rom.py validate` (see `tools/maxx_rom.py`) before burning EPROM.
4. For a worked 38-step demo, see `Cartridge/PROGRAMMING.md` §5.

Next: Chapter 2 — Opcode vocabulary

3 Chapter 2 — Opcode Vocabulary

This chapter lists every known Maxx Steele program opcode. Display names come from the internal ROM segment table at **`$F878`** (see Appendix B).

Opcodes are described in **hex order** within each group. For a one-page summary, see Quick reference.

Export machine-readable opcode JSON (`tools/maxx_rom.py`; output path is arbitrary, e.g. under `Cartridge/`):

```
python3 tools/maxx_rom.py opcodes Cartridge/opcodes.json
```

3.1 Motion and I/O opcodes (`$00–$10`)

Opcode	Display	Name	Operand
`$00`	L	Turn / drive left	Distance or angle
`$01`	F	Drive forward	Distance
`$02`	b	Drive reverse	Distance
`$03`	r	Turn / drive right	Angle
`$04`	Uu	Wrist up	Value
`$05`	Ud	Wrist down	Value
`$06`	Au	Arms up	Value
`$07`	Ad	Arms down	Value
`$08`	Cr	Claw rotate	Value
`$09`	Cc	Claw open/close	`$00`=open, `$01`=close
`$0A`	HL	Lamp	`$00`=off, `$01`=on
`$0B`	init	Home (all joints)	Must be `$00`
`$0C`	d	Delay	Seconds
`$0D`	Sn	Song number reference	Index
`$0E`	S	Speech (ROM phrase)	Phrase #
`$0F`	SS	Speech (shift mode)	Phrase #
`$10`	PS	Program speech capture	Slot #

Examples

```
01 14    ; forward
0A 01    ; lamp on
0C 0A    ; delay 10 seconds
82 3F    ; speak ROM phrase $3F
```

3.2 Extended opcodes (`$80–$FF`)

Opcodes with bit 7 set (`$80+`) are remapped through the display table for execution. The executor treats `$80` as an alias mapping to display index `$0C`.

Opcode	Display	Name	Operand
`$81`	PLAY	Play tune from music RAM	Tune # (`$0400` table)
`$82`	SPEE	Speak ROM phrase	Phrase #
`$83`	SS	Speak RAM phrase	Phrase # in `$0500`
`$84`	CLr	Clear speech phrase slot	Slot #
`$FE`	beg	Program begin marker	Display only
`$FF`	End	End of program	Must be `$FF` (pair `FF FF`)

3.3 Keypad-mapped opcodes

The internal ROM maps remote key scan codes to opcodes via table **$E6B5**. Faceplate names: `Transmitter/remote-keypad.md`. Documented mapping (from `maxx_internal_ROM.dsm`):

```
Key opcodes:  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
              41 46 43 13 84 82 81 83 80
```

Opcode	Typical keypad use
$41	Game-related step
$43	Click
$46	Motion step
$13	(mapped key — see learn/program mode)
$80–$84	Extended speech/music/clear group

In **Program mode**, pressing a key stores the mapped opcode and prompts for an operand byte.

3.4 Alphabetical index

Name	Opcode
arms_down	$07
arms_up	$06
claw_open_close	$09
claw_rotate	$08
delay	$0C
drive_forward	$01
drive_reverse	$02
end	$FF
home	$0B
lamp	$0A
play_tune	$81
song_number	$0D
speak_ram	$83
speak_rom	$82
speech_clear	$84
speech_program	$10
speech_rom	$0E
speech_shift	$0F
turn_left	$00
turn_right	$03
wrist_down	$05
wrist_up	$04

3.5 Unknown opcodes

If the executor encounters an undefined opcode, behavior is undefined in this documentation. Stick to opcodes listed above and validated with `tools/maxx_rom.py`.

Previous: Chapter 1 · **Next:** Chapter 3 — Motion and display

4 Chapter 3 — Programming Motion and Display

This chapter describes motion opcodes, the headlamp, and how the LED display shows program steps during entry and execution.

Distance and angle **operands are empirical** — the internal ROM scales them to motor timing. Values in the factory demo (e.g. `$14` forward, `$28` arm raise) are starting points, not documented engineering units.

4.1 Motion overview

Drive and joint motion opcodes `$00`–`$09` write timing values to motor control hardware mapped around **`$1600`** and **`$1C00`** (see Chapter 6).

The executor runs each step to completion (or until interrupted) before advancing the program pointer at `$0200`.

4.2 Drive opcodes

Opcode	Action	Demo cart examples
`$00`	Turn left	`00 05`
`$01`	Forward	`01 14`, `01 10`
`$02`	Reverse	`02 14`
`$03`	Turn right	`03 06`

Operands control how long or how far the drive system runs. Compare steps in `maxx_demo_ROM_532.dsm` with on-robot behavior when calibrating new values.

4.3 Arm and wrist opcodes

Opcode	Action	Demo example
`$04`	Wrist up	—
`$05`	Wrist down	`05 23`
`$06`	Arms up	`06 28`
`$07`	Arms down	—
`$08`	Claw rotate	`08 15`
`$09`	Claw open/close	`09 00` (open)

4.4 Home opcode `$0B`

Always use operand **`00`**:

```
0B 00    ; home — all joints to reference position
```

The demo cart homes before backing up at the end of the sequence.

4.5 Lamp opcode `$0A`

Operand	Effect
`$00`	Lamp off
`$01`	Lamp on

Demo sequence: `0A 01` … delay … `0A 00`.

4.6 LED segment display

During program entry and execution, the internal ROM fetches a **two-byte display code** from table **`$F878`** indexed by the current opcode (see `LDA $F878,Y` in `maxx_internal_ROM.dsm`). This drives the shift-register display at **`$1200`**.

Display names in Chapter 2 match that table (e.g. **L**, **F**, **PLAY**, **d**).

When program RAM is full, the display spells **FULL** (bytes at internal ROM label referencing `$037F` limit).

4.7 Programming techniques

1. **Home after motion blocks** — reduces accumulated joint error.
2. **Delay after every drive** — allows the chassis to settle before the next command.
3. **Lamp as status** — signal “busy” during long speech delays.
4. **Small operand sweeps** — test `$01 08`, `$01 10`, `$01 14` on your robot and record what works.

4.8 Known gaps

- Exact mm/degree mapping for operands is not documented in the internal ROM listing.
- Motor ramp profiles and limit switches are hardware-dependent; see `Mainboard/Schematic/` and `Mainboard/KiCAD/` (digitization in progress).

Previous: Chapter 2 · **Next:** Chapter 4 — Speech and music

5 Chapter 4 — Programming Speech and Music

This chapter describes how the Maxx Steele robot produces speech and music under program control.

Speech uses a parallel phoneme interface at **$1400**. The speech driver subroutine **$F3D5** clocks 4-bit nybbles to the synthesizer. Music uses byte pairs in RAM at **$0400** and an IRQ handler near **$F1D8**.

5.1 Speech overview

Mechanism	Opcode	Data source
ROM phrase table	$82, $0E	Internal ROM phoneme strings
RAM phrase table	$83	$0500 (copied from cartridge or recorded)
Program speech capture	$10	User-recorded slots
Clear phrase slot	$84	Clears RAM slot

5.2 ROM phrases ($82 / $0E)

Phrases are indexed phoneme strings in internal ROM. The driver at **$F3D5** outputs sounds with index **X** (see `JSR $F3D5` in `maxx_internal_ROM.dsm`).

Phoneme code tables reside at **$F567** / **$F4DB** (140 entries). Until the full **$F4DB** table is transcribed, treat phrase indices as opaque IDs.

Demo examples:

```
82 3F    ; "Ha ha ha ha ha" (ROM)
83 16    ; uses ROM index via RAM phrase indirection in demo — see cart listing
```

5.3 RAM phrases ($83)

Cartridges embed phrase data at offset **$81** (address **$A081** for demo cart). The bootstrap copies bytes to **$0500**.

Each phrase is a sequence of **16-bit phoneme tokens**; **$FF** pads unused slots.

Demo cart phrase table (from `maxx_demo_ROM_532.dsm`):

Slot	Content (abbrev.)
0	“Hello, I am Maxx Steele” / greeting tokens
1	“I am ready when you are”
2	“I am a great match for humans”
3	“Goodbye for now, have a good day”

Example program bytes:

```
83 10      ; speak RAM phrase 0 (demo uses operand $10 for table indexing — verify in .dsm)
0C 02    ; delay
83 00    ; another RAM phrase
```

Note: Demo listing uses operands $10, $00, $01, etc. — cross-check against your cart image with `tools/maxx_rom.py disasm`.

5.4 Music overview

Opcode **$81** plays a tune from the music RAM table at **$0400**.

Tune selection calls **$EF01** to resolve pointers for tune number in the operand (multiple call sites in internal ROM).

5.4.1 Music byte pairs

Stored in $0400 (and cartridge offset **$BB** / $A0BB):

```
70 12    ; note
70 11    ; note
38 0F
54 0D
E0 0F
00 00    ; end of tune
```

The music IRQ handler at **$F1D8** reads duration data from **$F15B** and frequency/note data from **$0400**.

Demo: `81 06` plays tune #6 (Reveille); `81 00` plays tune #0 at end of demo sequence.

5.5 Song number opcode $0D

References a song index for the internal song table (distinct from $81 play-tune in some keypad contexts). Used in learn/program flows.

5.6 Advanced notes

- **Phoneme authoring** requires transcribing $F4DB / $F567 — listed as future work in `Cartridge/PROGRAMMING.md` §10.
- **Speech error flag** (**SEr** in $02) — set when speech subsystem reports fault; demo cart entry sets related status.
- Face/speech hardware datasheets: `DataSheets/` (Sanyo LC3100/LC8100, ET9420 family).

Previous: Chapter 3 · **Next:** Chapter 5 — Internal ROM OS

6 Chapter 5 — Internal ROM Operating System

This chapter documents the robot's **8 KB masked ROM** (`$E000–$FFFF`): boot, operating modes, the main interpreter loop, device drivers, and how cartridge bootstrap stubs hand off to it.

Cartridge layout and authoring workflow remain in § Cartridge bootstrap below and in `Cartridge/PROGRAMMING.md`.

Primary sources: `Mainboard/Firmware/Assembly/maxx_internal_ROM.dsm` (R. Wind), `Mainboard/Firmware/Binary/Maxxrom.64`, `tools/maxxbas/patches.json` (simulator traps).

6.1 What the internal ROM is

At power-on the **MOS 6502** executes firmware in **`$E000–FFFF`**. This is the robot's only native execution environment — analogous to a C64 KERNAL, but specialized for:

- Cold/warm boot and RAM initialization
- Cartridge detection and table loading
- Keypad scan and **five operating modes** (`$0D` = 0–4)
- Bytecode fetch/decode from `$0200`
- Speech, music, motor, and display drivers
- Built-in games (mode 4)

User motion programs are **not** generally written in 6502. They are opcode/operand pairs in RAM, interpreted by this ROM. Cartridges supply a short **6502 bootstrap** plus data tables; they do not replace the interpreter.

6.2 Boot sequence

6.2.1 Reset and copyright check

Address	Role
`$E041`	**RESET** entry — `SEI`, init `$1E00` and display, set stack
`$E04F–$E058`	Compare ROM copyright at `$E000` with RAM mirror `$0100`
`$E05A`	**Cold start** — zero page, phrase RAM `$0500–$05FF` ← `$FF`, clear music
`$E06F`	**Warm start** — refresh copyright mirror, copy vector table

Cold start calls:

1. `$EF55` — zero music table `$0400–$04FF`
2. `$E582` — init program pointers and flags
3. `$E57B` — set program terminator at `$0180` ← `$FF`

Warm start copies the **37-byte vector table** from ROM `$E01C` into RAM **`$72–$96`**, then calls `$EEEE` (timer init).

6.2.2 RAM vector table ($72–$96)

Copied from ROM at boot. Key entries (initial values from listing):

ZP	Points to	Purpose
`$76`	`$E014`	NMI handler
`$78`	`$FDC8`	IRQ handler
`$7A`	`$E0CB`	Immediate-mode return (main loop)

ZP	Points to	Purpose
$7C–$85	$F1D8 region	Music IRQ jump table
$8C	$EE32	**Opcode dispatch** (JMP ($008C) at $EE2F)
$8E/$8F	$1000	Cartridge scan pointer (page/bank)
$90	$E598	Alternate dispatch
$92	$F3D8	Speech output vector
$94	$F8CE	**Game mode** entry

Full table: Appendix D — Zero-page registers.

6.2.3 Cartridge scan

After vectors are loaded, the ROM walks 4 KB boundaries from **$2000** through **$B000**:

1. Read 16-bit entry vector at cart base ($8E/$8F)
2. Compare bytes at offset +2 against ROM copyright template ((c) 1985 CBS Toys)
3. On match: store vector in $8E/$8F and **JMP ($008E)** into cartridge bootstrap
4. On no match: fall through to **$E0B6** (main loop without cart)

The factory demo cart at **$A000** is the reference bootstrap; see Cartridge bootstrap.

6.3 Main loop and mode dispatch

6.3.1 Entry: $E0B6

Whether or not a cartridge was found, execution eventually reaches **$E0B6**:

1. Display **"9"** on the LED ($ED4F / $ED7B)
2. Greet: *"Hello, I'm Maxx Steele."* ($F475, phrase $10) if speech enabled
3. Check stalled motors ($EDAF)
4. Set **$75 ← $80** (keypad ready), **$0D ← 0** (immediate mode)
5. Load startup tune 1 ($EF01)
6. Call $E905 (status/key housekeeping)
7. Dispatch on **$0D** at **$E0ED**

6.3.2 Mode variable $0D

Value	Mode	Display label	Entry
0	Immediate	—	$E154 — live key execution
1	Learn	LErn	$E161 — record key sequences
2	Program	Prog	$E346 — enter bytecode via keypad
3	Execute	run	$E434 — run program at $0200
4	Game	PLAy	JMP ($0094) → $F8CE

Modes 1–4 show a two-character LED label ($0D + 5 → index into display strings at $F684).

Mode changes re-enter through **$E0D1** (STA $0D then fall into the dispatch chain). The **GAME** remote key and clock/status paths also jump to $E0D1 with the new mode in A.

6.3.3 Shared busy flag: $2B

Many paths call **$EF63** first — a tight loop until **$2B = 0**. The listing marks $2B as a motor/speech **talkback** busy flag (motor controller confirms motion before the next command). The simulator bypasses this wait at $EF63 (patches.json).

6.3.4 Keypad input: $75 / $15

Decoded remote keys land in **$75** (bit 7 set when a key is pending). **$E617** polls the RF/matrix path; **$E6A4** copies the keycode to **$15** and clears the ready bit in $75.

Trap addresses for debugging: $E161 (learn loop), $E17B (immediate decode), $E617 (poll), $EE2F (opcode dispatch).

6.4 Mode-specific behavior

6.4.1 Immediate (mode 0) — $E154 / $E161 / $E17B

- Says *"I'm ready."* (phrase $20) when speech is on
- **$E161** loop: $E617 → test $15
- **$E17B**: key in A — WAIT ($0C), motion keys, PLAY, SPEECH, etc.
- Motion keys call **$EF2E** (drive) or **$EF40** (stop); PLAY resolves tune via **$EF01**
- **$E6B5** table maps keycodes 0–$13 to opcode bytes for learn/program paths

6.4.2 Learn (mode 1) — $E161 / $E574

- Init program pointer ($E574 sets $0200 terminator, $0F/$10 ← $0180)
- Prompt: *"Please teach me."* (phrase $1E)
- Records key sequences into $0200; CLEAR ($0E) calls **$E3A9**
- Shift/CLEAR keys move or prepend steps ($E2A2 region)

6.4.3 Program (mode 2) — $E346

- Shows current step number ($F64E)
- Keypad maps through **$E519** → opcode in **$11**, operand in **$13**
- **$E3F9** / **$E409** — step entry and RUN (ENTER) preview via **$F66C**
- OUT OF SPACE → *"Sorry, my circuits are full"* ($F40F)
- BEGIN ($13) decrements pointer; END markers $FF/$FE

6.4.4 Execute (mode 3) — $E434

- Sets run flag, walks **($0F,$10)** through $0200
- Each step: **$E519** → **$F66C** (execute one command) → **$E600** (wait for $75)
- **$FF** ends program; returns to immediate via **JMP ($007A)**
- Low battery: **$96** counter triggers *"I need energy, please recharge me."* during execute/game

6.4.5 Game (mode 4) — $F8CE

- Vector at **$94** → **$F8CE**
- Prompt: *"Please choose game"* (phrase $12)
- **Game 1** ($F8E9): reflex/timing style loop
- **Game 2** ($FAA4): extended play with difficulty selection ($F9E8)
- Win/lose speech via **$F40F**; replay prompt at **$FA67**

Game internals (scoring, IRQ timing at $FB8C–$FC6C) are not fully reverse-engineered in this archive.

6.5 Bytecode executor

6.5.1 One step: $F66C

Called from program mode preview, execute loop, and learn playback:

1. **$F737** — compute 1-based step number from $0F/$10
2. Load **$11** (opcode), **$13** (operand)
3. **$FF** → display **End**, exit; **$FE** → display **beg**
4. **$0B (HOME)** → four-byte init pattern from $F7F8–$F858 tables
5. Opcodes $80–$87 remap to table indices $0C–$13
6. Display two-character name from **$F878** table; show **$24/$25** as hex for extended opcodes
7. **JMP ($008C)** at **$EE2F** — dispatch to motion/speech/music handlers

6.5.2 Program pointer

Addr	Role
$0F/$10	Current step address in $0200 (starts $0180 = byte offset into table)
$11/$13	Current opcode / operand

Addr	Role
$24/$25	Executor working pointer (displayed for $8x opcodes)

Increment: **$E41A** (+2); decrement/BEGIN: **$E555**.

6.6 Device drivers

6.6.1 Display ($1200)

Address	Role
$ED4F	Shift LO bit to display chain
$ED7B	Send one byte to $1200
$ED48	Display prefix / spacing
$F684	Send null-terminated 2-char strings (mode labels, tune names)
$F878	Opcode → LED segment patterns (Appendix B)

Simulator bypasses serial wait loops at $ED5F–$EDA0 (patches.json).

6.6.2 Speech ($1400)

Address	Role
$F3D5	Speech output entry (JMP ($0092) → $F3D8)
$F460 / $F465	Clock nybbles to $1400
$F567 / $F4DB	Phoneme/duration tables (140 entries, index in X)
$F475	Say built-in ROM phrase by index (table at $F640)
$F40F	Say phrase with index in X (games, errors, prompts)

Built-in phrases $10–$20 include greetings, mode prompts, and game text (see listing at $F640).

6.6.3 Music

Address	Role
$EF01	Resolve tune pointer for operand (tune table in ROM)
$F15B	Note duration table (indexed 1–8)
$F1D8	Music IRQ handler region
$F0B8	Add note pair to $0400
$EF55	Clear music RAM

IRQ path decrements **$26**, **$28**, **$2A**, **$27** for note timing and game delays.

6.6.4 Motors ($1600, $1C00, $1E00)

Address	Role
$EF2E	Motor drive — Y = bank, A = command nibble
$EF40	Stop all motors (Y=$01, A=$6F)
$EF63	Wait for talkback ($2B = 0)
$EDAF	Stalled-motor check (main loop)
$EF76	Leave learn mode / motor cleanup

I/O bitfields at $1000, $1600, $1C00, and $1E00 are not fully documented; see Chapter 6.

6.7 Cartridge bootstrap

6.7.1 What a cartridge is

A **4 KB EPROM** maps into **$2000–$B000** in 4 KB steps. The factory demo sits at **$A000**.

A valid image supplies:

1. Entry vector and copyright header
2. A short **6502 bootstrap stub**
3. Program, speech, and music **data tables**

The stub copies tables into RAM and jumps to **$E0B6** — it does **not** replace the interpreter.

6.7.2 ROM layout ($A000 base)

Offset	Size	Content
$00	2	Entry vector (word), e.g. $A013
$02	17	Copyright: (c) 1985 CBS Toys
$13	code	Bootstrap stub (6502)
$35	table	Program bytecode
$81	table	Speech phrases (opcode $83)
$BB	table	Music notes (opcode $81)

Cartridge detection compares the copyright string at offset +2 against the internal ROM template.

6.7.3 Demo stub sequence

Entry at **$A013** (maxx_demo_ROM_532.dsm):

1. STA $02 / STA $03 — initialize status flags
2. Copy program from **$A035** → **$0200**
3. Copy phrases from **$A081** → **$0500**
4. Copy music from **$A0BB** → **$0400**
5. **JMP $E0B6** — enter internal ROM main loop

Expected entry prologue: A9 02 85 02 (LDA #$02 / STA $02). tools/maxx_rom.py validate checks this.

6.8 6502 vs bytecode — when to use which

Task	Language
Motion/speech demo sequence	Bytecode in $0200
Custom phrases/music in cart	Data tables + bootstrap
Patch OS behavior	6502 patch (advanced RE only)

Task	Language
New cartridge program	Bootstrap stub (minimal 6502) + bytecode tables
Debug firmware paths	`maxx simulate` + `patches.json` traps

Do not place arbitrary 6502 execution paths in the demo program area unless you fully understand cartridge mapping and internal ROM expectations.

6.9 Tools and simulation

Tool	Use
`Cartridge/PROGRAMMING.md` §9	Step-by-step cartridge authoring
`tools/maxx_rom.py`	`template`, `validate`, `disasm`
`tools/maxx simulate`	Patched ROM + program trace + traps
`Mainboard/Firmware/README.md`	ROM binary and listing index

```
python3 tools/maxx_rom.py template mycart.532
python3 tools/maxx_rom.py validate mycart.532
maxx simulate Cartridge/Examples/CBSDemo/Firmware/Binary/CBSDemo.532 --cycles 30000
```

EPROM type: Mitsubishi KM2365 family — `DataSheets/Mitsubishi-KM2365.pdf`.

6.10 Known gaps

- **Game internals** — scoring, `$A6–$AD` game state, IRQ paths `$FB8C–$FC6C` need deeper RE
- **Full zero page** — only programmer-facing locations are catalogued (Appendix D)
- **I/O bitfields** — `$1000`, `$1600`, `$1C00`, `$1E00` register layouts incomplete
- **Motor talkback** — `$EF63` / `$2B` protocol not fully traced to hardware
- **Phoneme tables** — `$F4DB` / `$F567` not transcribed to phoneme names
- **Complete subroutine catalog** — partial index in Appendix I; full `$E000–FFFF` map not yet mined

Previous: Chapter 4 · **Next:** Chapter 6 — I/O guide

7 Chapter 6 — Input/Output Guide

This chapter describes memory-mapped hardware, the remote keypad, cartridge slot, and the 27 MHz radio control link.

7.1 Memory map summary

Region	Address	Purpose
Zero page	`$0000–$00FF`	Flags, pointers, keypad state
Warm start	`$0100–$0103`	Copyright mirror
Stack	`$01FF ↓`	6502 stack
Program RAM	`$0200–$03FE`	Bytecode (2 bytes/step)
Music RAM	`$0400–$04FF`	Note pairs
Speech RAM	`$0500+`	Custom phrases
Timer / misc	`$1000`	I/O
Display	`$1200`	Shift-register LEDs
Speech chip	`$1400`	Parallel phoneme output
Motors	`$1600`, `$1C00`	Drive and joint timing
Cartridge ROM	`$2000–$B000`	4 KB slot stepping
Internal ROM	`$E000–$FFFF`	8 KB operating system

Full table: Appendix C.

7.2 Remote keypad (RF input)

The handheld transmitter uses a **COP411L** MCU scanning a row/column matrix. Key events are encoded as **27 MHz OOK** packets received by the robot superhet strip.

7.2.1 Matrix layout

	L7	L6	L5	L4	PK
D0	A	B	C	D	
D1	E	F	G	H	
L0	I	J	K	L	
L1	M	N	O	P	
L2	Q	R	S	T	
L3	U	V	W	X	
Gnd					Y

Faceplate labels (from): **U/1/2/3 DRIVE**,

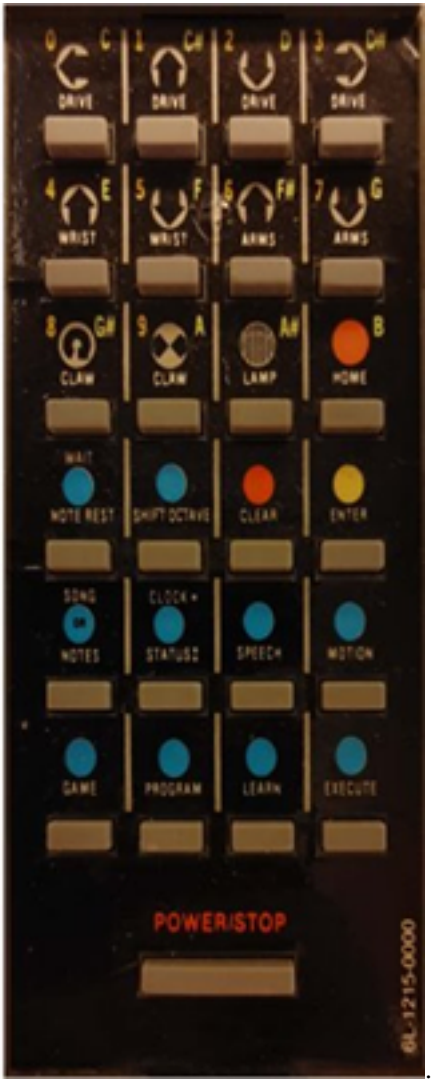
4/5 WRIST, **6/7 ARMS**, **8/9 CLAW**, **A LAMP**, **B HOME**, **NOTE REST** (WAIT), **SHIFT OCTAVE**, **CLEAR**, **ENTER**, **SONG/NOTES**, **CLOCK/STATUS**, **SPEECH**, **MOTION**, **GAME**, **PROGRAM**, **LEARN**, **EXECUTE**, **POWER/STOP** (**Y**).

Full key-to-matrix table: `Transmitter/remote-keypad.md`. Diagram: `Remote-Front.svg`.

Keyboard Matrix Addresses

Columns	L7	L6	L5	L4	P
Rows					
D0	A	B	C	D	
D1	E	F	G	H	
L0	I	J	K	L	
L1	M	N	O	P	
L2	Q	R	S	T	
L3	U	V	W	X	
Gnd					Y

Matrix wiring refs:



7.2.2 RF link (summary)

- Carrier: **27 MHz** on-off keying
- MCU clock: **455 kHz** ceramic resonator (IF reference)
- Bit period: **~1.55 ms**; packet repeat **~29 ms** (Power/Stop **~21 ms**)

Details: `Transmitter/transmitter-architecture.md`, `tools/rfcap/`.

Property	Value

7.3 Cartridge expansion

Property	Value
ROM size	4 KB per image
Mapping	`$2000`, `$6000`, `$A000`, … (4 KB steps)
Demo cart base	`$A000`
Required header	17-byte CBS Toys copyright at offset +2

Hardware EPROM: see `DataSheets/Mitsubishi-KM2365.pdf`.

7.4 On-board speech and face hardware

Speech synthesis ICs documented in `DataSheets/` include Sanyo LC3100/LC8100 and Eurotechnique ET9420 family parts. The CPU writes phoneme nybbles to **`$1400`**.

Face/display COP41xL subsystem: `National-COP41xL-Display-Motors.pdf` (refdes **U500** in archive naming).

7.5 CPU and support chips

Part	Role	Datasheet
MOS 6502	Main CPU	`MOS-6502.pdf`
COP420 family	Auxiliary MCU	`National-COP420.pdf`
M6116	Static RAM	`Mitsubishi-M6116.pdf`

CPU pinout notes (project doc): `Maxx-Steele-CPU-Pinout.pdf`.

7.6 Schematics

See Schematic diagram index for board-level drawings.

Previous: Chapter 5 · **Next:** Chapter 7 — UltraMaxx BASIC

8 Chapter 7 — UltraMaxx BASIC Language

This chapter describes **UltraMaxx BASIC** — a line-oriented programming language for authoring Maxx Steele cartridge programs without hand-editing bytecode. UltraMaxx BASIC is a **compiler language**, not an interpreter: each statement becomes one **(opcode, operand)** pair in the cartridge program table (see Chapter 1).

UltraMaxx BASIC is intentionally small. It is **not** Commodore 64 BASIC: there are no variables, no `IF/THEN`, no `GOTO`, and no arithmetic expressions. Think of it as a readable front end to the opcode vocabulary in Chapter 2.

Tools: `tools/maxx` (command line), `tools/maxxbas/` (Rust compiler).
Example sources: `ultramaxx.bas`, `cbsdemo.bas`, `hello.bas`.

8.1 Introduction

The factory demo cartridge (`CBSDemo.532`) stores its 38-step sequence as raw bytes at ROM address **`$A035`**. UltraMaxx BASIC lets you write the same sequence as text:

```
FORWARD 20
DELAY 4
ARMS UP 40
END
```

The compiler emits a standard **4096-byte** cartridge image (`.532`) with bootstrap stub, copyright string, program table, and optional phrase/music tables — the layout described in Chapter 5.

After compile, upload the image with PicoROM or burn EPROM. The robot runs the program through the normal internal ROM executor at **`$E000`**; the cartridge does not interpret BASIC at runtime.

8.2 Getting started

Add the toolchain (`tools/bin/`) to your path once:

```
export PATH="$(git rev-parse --show-toplevel)/tools/bin:$PATH"
```

Compile a source file:

```
maxx compile myprogram.bas -o mycart.532 --copyright ultramaxx
maxx validate mycart.532
```

Upload to a socketed PicoROM:

```
maxx upload myprogram.bas --device maxx_cart --copyright ultramaxx
```

Parse without writing output:

```
maxx check myprogram.bas
```

List program steps (JSON for simulators):

```
maxx list mycart.532 --json
```

8.3 Program format

8.3.1 Lines

A program is a text file, one statement per line. Blank lines are ignored.

Line numbers are optional and may be used for editor convenience (as in C64 listings). The compiler strips a leading numeric prefix:

```
10 DELAY 2
20 FORWARD 20
30 END
```

8.3.2 Comments

Two comment forms are accepted:

Form	Example
`REM` at start of line	`REM wait for motors`
`REM` after a statement	`FORWARD 20  REM short roll`
`#` to end of line	`LAMP ON  # headlamp`

Comments are not stored in the ROM image.

8.3.3 Statement syntax

- Keywords are **case-insensitive** (`delay`, `DELAY`, `Delay` are equivalent).
- Operands are unsigned integers **0–255** (decimal or `0x` hex).
- Multi-word keywords use spaces: `LAMP ON`, `ARMS UP`, `CLAW OPEN`.

8.3.4 Program terminator

Every program must end with **`END`**, which compiles to **`FF FF`**. If you omit `END`, the compiler appends it automatically.

8.4 Program size limits

Limit	Value	Notes
Steps per cartridge program	**38 pairs max**	Program ROM area `$A035`–`$A080` (76 bytes)
Operand range	0–255	One byte per step
Factory demo length	38 pairs	Fills the cartridge program slot
RAM program buffer	~255 pairs	`$0200` region; larger than cart ROM

If you exceed 38 pairs, the compiler reports **program too large**. Short programs (for example `hello.bas`) are valid; unused program bytes in ROM are filled with **`$FF`**.

The robot may accept longer sequences entered from the remote in Program mode, but a **standard 4 KB cart image** cannot ship more than 38 compiled steps without changing the fixed table layout.

8.5 UltraMaxx BASIC statements

Each statement maps to one bytecode pair. The **Byte** column shows the emitted `(opcode, operand)` in hex.

8.5.1 DELAY

Format: `DELAY` *seconds*

Action: Pause execution. Operand is delay time in **seconds** (opcode `$0C`).

Example	Bytes
`DELAY 2`	`0C 02`
`DELAY 10`	`0C 0A`

8.5.2 Drive: FORWARD, BACK, LEFT, RIGHT

Format: `FORWARD` *n* · `BACK` *n* · `LEFT` *n* · `RIGHT` *n*

Action: Drive or turn. Operand scaling is empirical — see Chapter 3.

Statement	Opcode	Demo example
`FORWARD 20`	`$01`	`01 14`
`BACK 20`	`$02`	`02 14`
`LEFT 5`	`$00`	`00 05`
`RIGHT 6`	`$03`	`03 06`

8.5.3 ARMS UP / ARMS DOWN

Format: `ARMS UP` *n* · `ARMS DOWN` *n*

Action: Raise or lower both arms (opcodes `$06` / `$07`).

Example	Bytes
`ARMS UP 40`	`06 28`
`ARMS DOWN 30`	`07 1E`

8.5.4 WRIST UP / WRIST DOWN

Format: `WRIST UP` *n* · `WRIST DOWN` *n*

Action: Wrist joint motion (opcodes `$04` / `$05`).

Example	Bytes
`WRIST DOWN 35`	`05 23`

8.5.5 CLAW ROTATE / CLAW OPEN / CLAW CLOSE

Format: `CLAW ROTATE` *n* · `CLAW OPEN` · `CLAW CLOSE`

Action: Claw rotate (`$08`), open (`$09`/`$00`), close (`$09`/`$01`).

Example	Bytes
`CLAW ROTATE 21`	`08 15`
`CLAW OPEN`	`09 00`
`CLAW CLOSE`	`09 01`

8.5.6 LAMP ON / LAMP OFF

Format: `LAMP ON` · `LAMP OFF`

Action: Head lamp (opcode `$0A`).

Example	Bytes
`LAMP ON`	`0A 01`
`LAMP OFF`	`0A 00`

8.5.7 HOME

Format: `HOME`

Action: Return all joints to reference position. Operand is always **`$00`**.

Example	Bytes
`HOME`	`0B 00`

8.5.8 PLAY

Format: `PLAY` *tune*

Action: Play a tune from the cartridge music table (opcode `$81`). Tune data lives at ROM **`$A0BB`** (copied to RAM **`$0400`**). See Chapter 4.

Example	Bytes
`PLAY 6`	`81 06`
`PLAY 0`	`81 00`

8.5.9 SAY

Format: `SAY` *phrase*

Action: Speak a **RAM phrase** by slot number (opcode `$83`). Phrase bytes must exist in the cartridge phrase table at **`$A081`** (copied to RAM **`$0500`**).

Example	Bytes	Factory demo text
`SAY 16`	`83 10`	“Hello, I am Maxx Steele”
`SAY 0`	`83 00`	“I am great, and you”

UltraMaxx BASIC does **not** yet compile quoted strings (`SAY "HELLO"`) into phoneme nybbles. Use **`SPEAK`** for built-in ROM speech, or compile with **`--tables-from`** to copy phrase bytes from a reference ROM (see §Phrase and music tables below).

8.5.10 SPEAK

Format: `SPEAK` *phrase*

Action: Speak a **ROM phrase** by index (opcode `$82`). Uses built-in phoneme data in internal ROM — no cart phrase table required.

Example	Bytes	Notes
`SPEAK 63`	`82 3F`	Laugh (factory demo)

8.5.11 END

Format: `END`

Action: Terminate the program. Both bytes must be **`$FF`**.

Example	Bytes
`END`	`FF FF`

8.6 Statement summary table

UltraMaxx BASIC	Opcode	Operand
`DELAY` *n*	`$0C`	seconds
`FORWARD` *n*	`$01`	distance
`BACK` *n*	`$02`	distance
`LEFT` *n*	`$00`	distance/angle
`RIGHT` *n*	`$03`	angle
`WRIST UP` *n*	`$04`	value
`WRIST DOWN` *n*	`$05`	value
`ARMS UP` *n*	`$06`	value
`ARMS DOWN` *n*	`$07`	value
`CLAW ROTATE` *n*	`$08`	value
`CLAW OPEN`	`$09`	`$00`
`CLAW CLOSE`	`$09`	`$01`
`LAMP ON` / `OFF`	`$0A`	`$01` / `$00`
`HOME`	`$0B`	`$00`
`PLAY` *n*	`$81`	tune index
`SPEAK` *n*	`$82`	ROM phrase
`SAY` *n*	`$83`	RAM phrase slot
`END`	`$FF`	`$FF`

8.7 Copyright strings

The compiler writes a 17-byte copyright field at ROM offset **`$02`**. Choose with **`--copyright`**:

Value	String	Typical use
`ultramaxx`	`(c) UltraMaxx`	Community cartridges
`cbs`	`(c) 1985 CBS Toys`	CBS factory branding

Cartridge detection in internal ROM compares this field during warm start.

8.8 Phrase and music tables

Programs that use **`SAY`** or **`PLAY`** need matching data in the cart tables at **`$A081`** and **`$A0BB`**. The compiler fills these regions with **`$FF`** / **`$00`** unless you supply a reference image:

Example (`ultramaxx.bas`, tables from `UltraMaxx.532`):

```
maxx compile ultramaxx.bas -o UltraMaxx.bas.532 \
  --copyright ultramaxx \
  --tables-from Cartridge/Examples/UltraMaxx/Firmware/Binary/UltraMaxx.532
```

This copies phrase and music bytes from the reference ROM while replacing the program table with your compiled source. The full factory demo in `ultramaxx.bas` compiles **byte-identical** to stock `UltraMaxx.532` when tables are copied from that file.

Custom speech authoring (phoneme encoding) remains future work — see Chapter 4 and `Cartridge/PROGRAMMING.md` §10.

8.9 Sample programs

8.9.1 Minimal program

```
DELAY 1
FORWARD 20
```

```
LAMP ON
DELAY 3
LAMP OFF
HOME
END
```

Source: `hello.bas`.

8.9.2 Factory demo (38 steps)

The complete CBS/UltraMaxx demonstration sequence — speech, motion, lamp, tunes, home, and backup — is available as:

Branding	Source
CBS Toys	`cbsdemo.bas`
UltraMaxx	`ultramaxx.bas`

Both sources compile to the same motion program; only the copyright string differs.

8.10 Compiler errors

UltraMaxx BASIC reports line-numbered errors at compile time (not on the robot display).

Message	Cause
`empty program`	No statements in file
`unknown statement`	Keyword not in vocabulary (e.g. `GOTO`, `PRINT`)
`operand … out of range 0..255`	Numeric operand too large
`program too large`	More than 38 pairs
`DELAY requires one operand`	Missing or extra operands
`LAMP requires ON or OFF`	Invalid lamp sub-keyword
`validation failed`	Emitted image failed cart structure checks

Use **`maxx check`** *file* to validate syntax without writing a `.532` file.

8.11 What UltraMaxx BASIC is not

The following Commodore 64 BASIC features are **intentionally absent**:

C64 BASIC	UltraMaxx BASIC
Variables (`A`, `X$`)	Not supported
`IF` / `THEN`	Not supported
`GOTO` / `GOSUB`	Not supported
`FOR` / `NEXT`	Not supported
Arithmetic expressions	Not supported
`INPUT` / `READ` / `DATA`	Not supported
String functions	Not supported
Inline interpreter	Not supported — compile only

Control flow is **strictly linear**: the executor runs steps in order from `$0200` until **`END`**.

8.12 Relationship to other chapters

Topic	See
Bytecode rules, modes, RAM limits	Chapter 1
Opcode hex reference	Chapter 2
Motion operand tuning	Chapter 3
Speech phonemes, music bytes	Chapter 4
Bootstrap stub, ROM map	Chapter 5
Cartridge authoring workflow	`Cartridge/PROGRAMMING.md`
PicoROM upload	`Cartridge/Examples/UltraMaxx/PICOROM.m`

8.13 Quick command reference

All commands are provided by `tools/maxx` (see `tools/maxxbas/` for the Rust implementation).

Command	Purpose
`maxx compile` *file.bas*	Build `.532` image
`maxx check` *file.bas*	Syntax check
`maxx validate` *file.532*	Verify cart structure
`maxx list` *file.532*	Human-readable step list
`maxx list --json`	Machine-readable trace (simulators)
`maxx upload` *file*	Compile (if needed) + PicoROM
`maxx simulate` *file*	Step preview (`tools/maxxbas/`; add `--gui` for interactive playback)

Previous: Chapter 6 — Input/output guide · **Next:** Appendices

9 Appendices

Reference tables for the Maxx Steele Programmer's Reference Guide.

9.1 A — Opcode abbreviations

See Quick reference for the full hex table. Names match `tools/maxx_rom.py` `OPCODES` dict.

9.2 B — Display segment table

Internal ROM table **`$F878`** maps opcode indices to two-character LED segment patterns. Documented display names:

Index	Display	Opcode context
—	L, F, b, r	Drive
—	Uu, Ud, Au, Ad, Cr, Cc	Joints
—	HL, init, d	Lamp, home, delay
—	PLAY, SPEE, SS, CLr	Extended
—	End, beg	`$FF`, `$FE`

Full glyph bitmaps are embedded in `maxx_internal_ROM.dsm` near label `FULL` / `$F878` references.

9.3 C — Memory map

Region	Address	Size	Purpose
Zero page	`$0000–$00FF`	256 B	Flags, pointers
Warm start	`$0100–$0103`	4 B	Copyright mirror
Stack	`$01FF ↓`	—	6502 stack
Program RAM	`$0200–$03FE`	510 B	Bytecode
Music RAM	`$0400–$04FF`	256 B	Note pairs
Speech RAM	`$0500+`	—	Phrases for `$83`
I/O	`$1000`	—	Timer
Display	`$1200`	—	LED shift register
Speech	`$1400`	—	Phoneme parallel port
Motors	`$1600, $1C00`	—	Motor drive
Cartridge	`$2000–$B000`	4K slots	Plug-in ROM
Internal ROM	`$E000–$FFFF`	8 KB	OS / interpreter

9.4 D — Zero-page registers

9.4.1 Status and mode

Addr	Name	Purpose
`$02`	Status A	Speech error, backoff, power, drive flags (Appendix E)
`$03`	Status B	User control, speech enable, execute gating
`$04`	Reveille flag	Set by clock path; cleared at `$EE32`
`$0D`	Mode	0=immediate, 1=learn, 2=program, 3=execute, **4=game**
`$0E`	Entry state	Non-zero during multi-key opcode entry

Addr	Name	Purpose
$0F/$10	Program pointer	Address of current step in $0200 (beg = $0180)
$11/$13	Current op/operand	Fetched by $E519 for executor
$15	Keycode	Last decoded keypad value from $E617
$24/$25	Executor pointer	Working address; displayed for extended opcodes
$2B	Talkback busy	Non-zero while motors/speech pending; $EF63 waits
$26	Timer prescale	IRQ countdown; reset to $F4 at $E99
$27	Second timer	Game delays, enter-key timeout
$39–$3C	Music state	Pointers/flags during note entry ($EF76 region)
$56	IRQ phase	Selects handler in IRQ jump table
$57	Speech timing	Used during $1400 nybble output
$5B	Speech status	Error/ready flags for LC8100 path
$67–$6A	Motor timing	Decremented in IRQ; init to $FF at warm start
$72–$96	Vector table	Copied from ROM $E01C at boot (see below)
$74	Tune index	Last PLAY operand; power-down tune selection
$75	Keypad buffer	Bit 7 = key ready; $E600 waits for match
$8E/$8F	Cart pointer	16-bit cartridge scan address (page in $8F)
$96	Energy counter	Decremented in IRQ; triggers low-battery speech

9.4.2 Vector table ($72–$96, from ROM $E01C)

ZP	Initial target	Role
$72/$73	$0082	Reserved / table header
$76/$77	$E014	NMI vector
$78/$79	$FDC8	IRQ vector
$7A/$7B	$E0CB	Return to main loop (immediate)
$7C–$85	$F1D8…	Music IRQ dispatch table
$8C/$8D	$EE32	Opcode dispatch ($EE2F)
$8E/$8F	$1000	Cartridge entry pointer
$90/$91	$E598	Alternate handler vector
$92/$93	$F3D8	Speech output vector
$94/$95	$F8CE	Game mode entry

9.5 E — Status bits

Bit	Label

9.5.1 `$02` (from `tools/maxx_rom.py`)

Bit	Label
0	SEr (speech error)
1	Ebof (enable backoff)
2	PDon (power down on)
3	Edof (enable drive off)
4	SPon (speech on)

Keypad toggles (internal ROM): row 0 → SEr, PAr; row 2 → Edon, Edof; row 3 → UCon, UCof; row 4 → Ebon, Ebof; row 5 → SPon, SPof.

9.5.2 `$03`

Bit	Label
0	UCon (user control on)
1	SPon (speech enable)

Demo bootstrap: `$02` ← `$02`, `$03` ← `$82`.

9.6 F — Music duration table

Note durations for the music IRQ path are read from **`$F15B`** (indexed 1–8 in `maxx_internal_ROM.dsm`). Frequency/note bytes live in **`$0400`**.

Complete note name → byte mapping is not yet transcribed in this archive.

9.7 G — Cartridge file layout

File offset = cartridge address − `$A000` for demo cart base.

File offset	Addr	Content
`$0000`	`$A000`	Entry vector
`$0002`	`$A002`	Copyright (17 bytes)
`$0013`	`$A013`	Bootstrap code
`$0035`	`$A035`	Program table
`$0081`	`$A081`	Phrase table
`$00BB`	`$A0BB`	Music table

9.8 H — `tools/maxx_rom.py` commands

```
python3 tools/maxx_rom.py disasm PATH [--compare-dsm DSM]
python3 tools/maxx_rom.py validate PATH
python3 tools/maxx_rom.py template OUTPUT.532
python3 tools/maxx_rom.py opcodes OUTPUT.json
```

9.9 I — Internal ROM entry points

Curated catalog from `maxx_internal_ROM.dsm`. See Chapter 5 for boot flow and mode diagrams.

9.9.1 Boot and initialization

Address	Role
$E041	RESET entry
$E05A	Cold start — zero page, clear RAM tables
$E06F	Warm start — copy vectors $E01C → $72
$E08E–$E0B3	Cartridge scan and JMP ($008E)
$E574	Init learn pointer, $0200 ← $FF
$E582	Init program pointers (cold start)
$E57B	Set $0180 ← $FF terminator
$EEEE	Timer byte init

9.9.2 Main loop and modes

Address	Role
$E0B6	Main loop entry (post-cart or no cart)
$E0D1	Mode change — STA $0D, re-dispatch
$E0ED	Mode dispatch on $0D
$E154	Immediate mode entry
$E161	Learn/immediate keypad loop
$E17B	Immediate key decode (key in A)
$E346	Program mode loop
$E434	Execute mode runner
$E905	Status housekeeping (called each loop)
$F8CE	Game mode entry (JMP ($0094))
$F8E9	Game 1 main
$FAA4	Game 2 main

9.9.3 Keypad and program I/O

Address	Role
$E3A9	CLEAR program function
$E3EC	Delay / wait for flag change
$E409	RUN preview (ENTER in program mode)
$E41A	Increment program pointer (+2)
$E519	Fetch opcode/operand to $11/$13
$E555	Decrement pointer (BEGIN)
$E600	Wait until $75 matches A
$E617	Poll keypad → $15
$E6A4	Key poll wrapper (motors, RF)
$E6B5	Keycode → opcode translation table
$E9FE	Keypad input helper (games, music entry)

Address	Role

9.9.4 Bytecode executor

Address	Role
`$EE2F`	`JMP ($008C)` — opcode dispatch
`$EE32`	Dispatch entry (Reveille, execute paths)
`$F64E`	Program step number → display
`$F66C`	Execute one program command
`$F737`	Compute step number from `$0F`/`$10`

9.9.5 Display

Address	Role
`$ED48`	Display prefix
`$ED4F`	Shift LO bit to `$1200`
`$ED7B`	Send byte to display
`$EDAF`	Stalled-motor check
`$F684`	Send 2-char string to display
`$F878`	Opcode → segment pattern table

9.9.6 Speech

Address	Role
`$F3D5`	Speech output entry (`JMP ($0092)`)
`$F3D8`	Phoneme output (index X)
`$F40F`	Say ROM phrase by index (X)
`$F460` / `$F465`	Clock nybbles to `$1400`
`$F475`	Say built-in phrase (index X)
`$F4DB`	Phoneme duration table (140 entries)
`$F567`	Phoneme code table

9.9.7 Music

Address	Role
`$EF01`	Resolve tune pointer for operand
`$EF55`	Clear `$0400` music table
`$EF76`	Leave learn / music cleanup
`$F0B8`	Append note to `$0400`
`$F15B`	Note duration table
`$F1D8`	Music IRQ handler region

9.9.8 Motors

Address	Role
`$EF2E`	Motor drive command
`$EF40`	Stop motors
`$EF63`	Wait for talkback (`$2B` = 0)

9.10 J — Messages and errors

Message / flag	Cause
FULL	Program exceeds `$037F` / RAM limit
SEr	Speech error status bit
`validate` copyright mismatch	Cart missing `(c) 1985 CBS Toys`
Missing `FF FF`	Program table not terminated

9.11 K — Bibliography

- R. Wind — `maxx_internal_ROM.dsm`, `maxx_demo_ROM_532.dsm` (2002–2006)
- Factory manual — `Chassis/Manual/MaxxSteeleReferenceGuide.pdf`
- This repository — https://github.com/webaugur/Maxx-Steele-1984-Robot
- C64 Programmer's Reference Guide — structural inspiration (Commodore, 1982)

9.12 L — Glossary

Term	Meaning
Bytecode	Opcode/operand pairs interpreted by internal ROM
Bootstrap	Short 6502 stub in cartridge that loads RAM tables
OOK	On-off keying RF modulation (27 MHz)
IF	Intermediate frequency (455 kHz in transmitter/receiver)
Phoneme	Speech sound unit; 4-bit codes to `$1400`
Refdes	Schematic reference designator (U1, U400, …)
`.532`	4 KB cartridge image file extension used in archive

10 Maxx Steele Quick Reference Card

Single-page summary. Full detail in chapters 1–7 and Appendices.

10.1 Modes (`$0D`)

Val	Mode
0	Immediate
1	Learn
2	Program
3	Execute

10.2 Key zero page

Addr	Use
`$02`	Status A
`$03`	Status B
`$0D`	Mode
`$0F/$10`	Program step pointer
`$0200`	Program RAM base
`$0400`	Music RAM
`$0500`	Speech RAM

10.3 Motion opcodes

Hex	Disp	Action
00	L	Left
01	F	Forward
02	b	Reverse
03	r	Right
04	Uu	Wrist up
05	Ud	Wrist down
06	Au	Arms up
07	Ad	Arms down
08	Cr	Claw rotate
09	Cc	Claw open/close
0A	HL	Lamp
0B	init	Home (`00`)
0C	d	Delay (sec)

10.4 Speech / music

Hex	Disp	Action
0E	S	Speech ROM
0F	SS	Speech shift
10	PS	Program speech
81	PLAY	Play tune
82	SPEE	Speak ROM

Hex	Disp	Action
83	SS	Speak RAM
84	CLr	Clear phrase
FE	beg	Marker
FF	End	End (`FF FF`)

10.5 Cartridge header (`$A000`)

Off	Content
+0	Entry vector
+2	`(c) 1985 CBS Toys`
+$13	Bootstrap
+$35	Program
+81|$Phrases$||+BB	Music

10.6 Common sequences

```
0C nn    delay nn seconds
0B 00    home
0A 01    lamp on
0A 00    lamp off
FF FF    end program
```

10.7 UltraMaxx BASIC (Ch 7)

Statement	Example
Delay	`DELAY 2`
Drive	`FORWARD 20` `BACK 20` `LEFT 5` `RIGHT 6`
Joints	`ARMS UP 40` `WRIST DOWN 35` `CLAW ROTATE 21` `CLAW OPEN`
Lamp	`LAMP ON` / `LAMP OFF`
Home	`HOME`
Music	`PLAY 6`
Speech	`SPEAK 63` (ROM) · `SAY 0` (RAM slot)
End	`END`

Cart limit: 38 steps max. **Not supported:** variables, `IF`, `GOTO`.

CLI (`tools/maxx`):

```
maxx compile FILE.bas -o FILE.532
maxx upload FILE.bas --device maxx_cart
```

10.8 Tools

Binaries: `tools/maxx`, `tools/maxx_rom.py` (shell wrapper `tools/bin/maxx-rom`).

```
maxx compile FILE.bas -o FILE.532
maxx-rom validate FILE.532
maxx-rom template FILE.532
```

11 Schematic Diagram Index

The Commodore 64 Programmer's Reference included a fold-out schematic. This index points to equivalent materials in the Maxx Steele archive.

11.1 Main logic board

Asset	Path	Status
Raster schematic (enhanced)	`Mainboard/Schematic/MaxxSteele_Schematic_enh-v1.1.png`	Available
SVG (v1.1)	`Mainboard/Schematic/v1.1/MaxxSteele_Schematic.svg`	Available
KiCad project	`Mainboard/KiCAD/`	Planned — digitization in progress

6502 CPU, RAM (M6116), cartridge interface, speech and motor glue.

11.2 Transmitter (remote)

Asset	Path	Status
KiCad schematic + PCB	`Transmitter/KiCAD/Transmitter-27MHz.kicad_pro`	Active
RE notes	`Transmitter/ReverseEngineering/`	Available
BOM	`Transmitter/transmitter-bom.md`	Available

COP411L (U1), 455 kHz clock, 27 MHz OOK stage.

11.3 Receiver (robot RF)

Asset	Path	Status
KiCad schematic	`Receiver/KiCAD/Receiver-27MHz.kicad_pro`	Schematic only
PCB layout	—	Not yet in archive

11.4 Cartridge

Asset	Path	Status
KiCad (CBSDemo)	`Cartridge/Examples/CBSDemo/KiCAD/CBSDemo.kicad_pro`	Active
KiCad (UltraMaxx)	`Cartridge/Examples/UltraMaxx/KiCAD/` (same PCB as CBSDemo)	Active
PicoROM (UltraMaxx U1)	`Cartridge/Examples/UltraMaxx/PICOROM.md`	Active

11.5 Face / speech / display

Asset	Path	Status
KiCad	`Face/KiCAD/`	Planned
Datasheets	`DataSheets/`	LC3100, LC8100, ET9420, COP41xL

11.6 Power module

Asset	Path	Status
KiCad	`Power/KiCAD/`	Planned

11.7 Paddle mirror accessory

Asset	Path	Status
Photo	`PaddleMirror/Photos/`	Available
Schematic	—	**Needed**

11.8 Chip cross-reference

Indexed datasheets with refdes where known: `DataSheets/README.md`.

ABOUT THE MAXX STEELE . . . ROBO FORCE TECHNICAL MANUAL . . .

Action figure compatibility . . . Advanced Robotic Speech and Sound . . . and high tech battle capabilities make the Maxx Steele Robot Force the most advanced robotic toy in its class for home, play, and educational use.

The MAXX STEELE ROBO FORCE TECHNICAL MANUAL tells you everything you need to know about your Maxx Steele robot. The perfect companion to your Robot Force Commander's Guide, this manual presents detailed information on everything from laser blasters to advanced machine language techniques. This book is a must for everyone from the beginner to the advanced robotic engineer.

For the beginner, the most complicated topics are explained with many sample programs and an easy-to-read wiring style. For the advanced robotic engineer, this book has been subjected to heavy pre-testing with your needs in mind. And it's designed so that you can easily get the most out of your Robot Force capabilities.
engineer.

For the beginner, the most complicated topics are explained with many sample programs and an easy-to-read wiring style. For the advanced robotic engineer, this book has been subjected to heavy pre-testing with your needs in mind. And it's designed so that you can easily get the most out of your Robot Force capabilities.

maxx steele
ROBO FORCE

Maxx Steele Robot Force, Inc. — Robot Systems Division.
1301 Western Avenue, Cincinnati, Ohio 45203

$19.95/82056

ISBN 0-678-12345-6